Save Point

QuillFin: Production Plan

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

QuillFin is a Save Point application: notes, AI-generated quizzes and a practice-problem tracker, each scheduled with FSRS, working offline on phone and desktop and in the browser. This document is the reference for turning the current local-only Flutter app into that product. Decisions marked **(decided)** were made by the product owner on 2026-10-27.

---

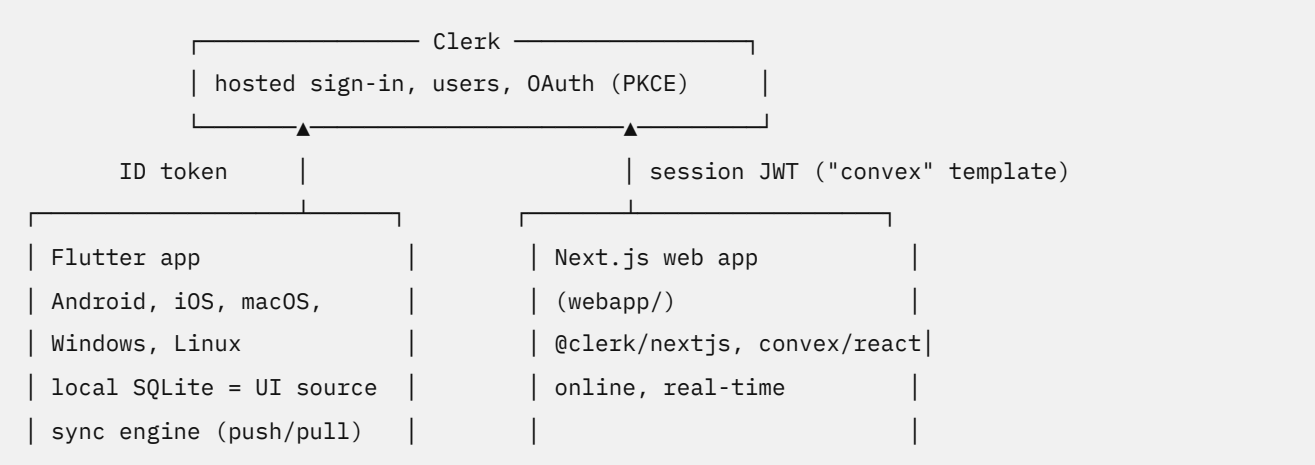
1. Product shape

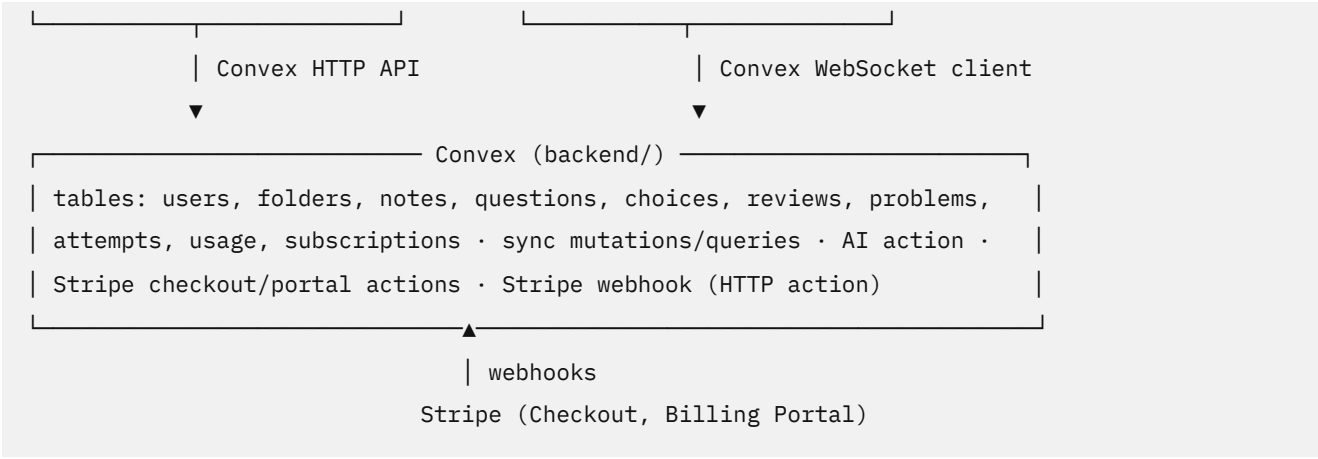
	Free	Pro
Notes, folders, file import	Unlimited	Unlimited
Problem tracker, FSRS, reminders	Yes	Yes
Sync across devices	Yes	Yes
AI question generation	20 notes / month on QuillFin's key, or unlimited with your own key	500 notes / month on QuillFin's key
Price	$0	Monthly or yearly (prices set in Stripe)

(decided) Free tier plus a Pro subscription. The limits above are placeholders that live in one place (`backend/convex/plans.ts`).

---

2. Architecture





(decided) Flutter for phone and desktop, plus a separate Next.js web app, sharing one Convex backend.

Repository layout

```
/                Flutter app (unchanged location, so CI and packaging keep working)
backend/         Convex project (TypeScript): schema, functions, tests (vitest + convex-
test)
webapp/          Next.js app (App Router, TypeScript, Tailwind)
package.json     npm workspaces: backend, webapp
Docs/            all documentation, including setup_guide.md
```

Why these integration choices

- **Clerk on Flutter: OAuth 2.0 authorization code + PKCE as a public client**, through the system browser (`flutter_web_auth_2`): custom URL scheme `com.savepoint.quillfin://callback` on Android/iOS/macOS, a loopback `http://localhost:<port>/callback` on Windows/Linux. Clerk's official Flutter SDK is beta and has no Windows/Linux support; this flow is a documented Clerk feature that works on every native platform and keeps passwords out of the app.
- **Convex auth**: `auth.config.ts` lists two providers. The web app's Clerk session JWT (template `convex` , `aud = "convex"`), and the Flutter app's Clerk OIDC ID token (`aud = <OAuth client id>`). Both have `iss = <Clerk Frontend API URL>` . Every function reads the user from `ctx.auth.getUserIdentity().subject` (the Clerk user id).
- **Convex from Flutter: the official HTTP API** (`POST /api/query` , `/api/mutation` , `/api/action` with `Authorization: Bearer <token>`). Offline-first sync is request/response by nature, so no WebSocket client is needed, and it avoids depending on a community package.
- **Stripe: Checkout + Billing Portal + webhook**, all server-side in Convex. Apps open the returned URL; nothing card-related touches the app. The webhook (`/stripe/webhook` HTTP action) verifies the signature and writes `subscriptions` ; entitlement is always read from Convex.
- **iOS caveat**: outside the US storefront, Apple requires In-App Purchase for digital subscriptions bought inside an iOS app. Until IAP is added, the iOS build hides "Upgrade" and shows "Manage your plan at quillfin.app" text only, as a reader app. Android and desktop can link to Stripe Checkout. Revisit before App Store submission.

3. Offline-first sync

(decided) The local SQLite database stays the source of truth for the UI, and works with no connection. Sync runs on launch, on resume, after every local write (debounced 2s) and every 5 min while open.

- Every synced row gets a client-generated `uid` (UUID v4), an `updated_at`, a nullable `deleted_at` tombstone, and a local-only `dirty` flag. Local integer ids and foreign keys stay as they are; the server stores parent `uid`s.
- **Push**: `sync:push` takes batches of dirty rows per table (≤200 per call). The server upserts by (`userId`, `uid`) with **last-writer-wins on** `updatedAt`, and stamps a per-user monotonic `seq`.
- **Pull**: `sync:pull(since: seq)` returns rows with `seq > since`, paged, including tombstones. The client applies them (skipping its own still-dirty rows that are newer) and stores the highest `seq` as its cursor.
- **Append-only tables** (`question_history`, `problem_attempts`) never conflict: they're inserted once and never updated.
- **FSRS cards** are fields on their row, so LWW applies. Reviewing the same card on two offline devices keeps the later review's state, while both history rows survive. A later "rebuild card from history" job could reconcile exactly (FSRS is replayable from its review log).
- **First sign-in on a device that already has data**: upload all local rows as dirty, the same path as any other push. Signing out keeps local data but stops syncing; "Delete local data" is offered.
- Server-side limits: max note size 200 KB, max 50k rows per user per table, with a clear error surfaced in Settings > Sync.

4. Visual identity (placeholder)

Logos, colours and fonts are "coming soon". Until then the placeholder below lives in exactly two token files (`lib/theme/brand.dart` and `webapp/app/brand.css`) so the real brand is a token swap:

- **Name**: QuillFin, with a capital F. "QuillFin by Save Point" on the sign-in and about screens.
- **Mark**: a quill nib whose feather curves into a fin. Geometric, one colour, readable at 16 px (`assets/brand/quillfin-mark.svg`).
- **Palette** (light / dark):

Token	Light	Dark	Use
`ink`	`#12212E`	`#E8EEF2`	text
`inkSoft`	`#4C5C69`	`#A9B6C0`	secondary text
`paper`	`#F7F6F2`	`#0D1821`	background
`surface`	`#FFFFFF`	`#15232F`	cards, sheets
`surfaceTint`	`#EEF3F2`	`#1C2E3B`	quiet fills
`fin` (primary)	`#0F8A7E`	`#3CC3B3`	buttons, links, selection

Token	Light	Dark	Use
`deep`	`#0E3B4F`	`#9ED8E6`	headers, emphasis
`quill` (accent)	`#F2B33D`	`#F5C561`	highlights, "due"
`danger`	`#C2412D`	`#FF8A73`	errors
`success`	`#2F9E5C`	`#6DD69A`	solved, correct

All text/background pairs meet WCAG AA at body size.

- **Type**: **Outfit** (display: headings, numbers, buttons) and **Inter** (body), both SIL OFL, bundled in the app and loaded with `next/font` on web.
- **Shape**: 14 px card radius, 12 px controls, 1 px hairline borders, soft low shadows only on raised surfaces, 4/8-pt spacing grid.
- **Dark mode**: follows the system, with a manual override in Settings.

---

5. Layouts per screen size

Breakpoints follow Material 3 window size classes, by **logical width**, identically in Flutter (`lib/layout/breakpoints.dart`) and on web (Tailwind `sm/md/lg/xl/2xl` remapped to the same numbers):

Class	Width	Typical devices	Navigation	Content
Compact	< 600	phones portrait (360-430)	bottom nav bar, 5 items	single pane, 16 px gutters
Medium	600-839	foldables, small tablets portrait, phones landscape	navigation rail (icons + labels)	single pane, 24 px gutters, max 640 px reading width
Expanded	840-1199	tablets landscape, small laptop windows	rail, FAB in rail	**list-detail two-pane** (list 360 px, detail fills)
Large	1200-1599	laptops, desktops	permanent sidebar (240 px, labels, account footer)	two-pane, detail max 760 px centred
Extra-large	≥ 1600	big monitors	sidebar	three-pane where useful (folder tree / list / detail), content max 1440 px

Height matters too: when the height is under 480 (phone landscape), the bottom bar becomes a rail and sheets become side panels.

Per screen

- **Today**
  - Compact: hero card, then a 2×2 stat grid, problems due, a 7-day forecast.

    - Medium: stats become a 4-across row.
    - Expanded and up: two columns, with hero, stats and forecast on the left (60%) and "Problems to re-solve" plus "Recently reviewed" on the right.
- **Notes**
    - Compact: folder list, then a folder page, then the note editor full-screen.
    - Expanded: folders and notes list on the left, editor on the right; the selected note stays open while browsing.
    - Extra-large: folder tree / note list / editor.
    - The editor has a max 72-character line length with centred text on wide panes.
- **Tracker**
    - Compact: filter chips scroll horizontally; cards; full-screen detail.
    - Expanded: list on the left, problem detail on the right; the "Log attempt" sheet becomes a right-side dialog.
    - Large: adds a sortable table view toggle (name, difficulty, status, next due, recall, attempts).
- **Quiz session**: always a focused single column, max 640 px, centred.
    - Compact: rating buttons pinned to the bottom, within thumb reach.
    - Wider than compact: rating buttons sit under the answer.
- **Settings / Account / Billing**
    - Compact: grouped list, then detail pages.
    - Expanded and up: section list on the left, section on the right (System Settings style).
- **Sign-in**
    - Compact: full-screen brand and button.
    - Large: split screen, with brand art on the left and sign-in on the right.

Desktop friendliness (Flutter desktop and web)

- **Keyboard shortcuts**:
    - `Ctrl/⌘+N` new note
    - `Ctrl/⌘+K` search
    - `Ctrl/⌘+1…5` switch tabs
    - in a quiz, `Enter` submits and `1-4` rate
    - `Esc` closes panels
    - `?` lists all shortcuts
- **Pointer**: hover states on every interactive row, right-click context menus (rename, move, delete), and drag notes between folders.
- **Window**: minimum 360×560. Window size and position are remembered, and resizing re-flows between classes without losing state.
- **Text**: respects the system text scale up to 200%; layouts tested at 1.0, 1.3 and 2.0.
- **Touch targets**: ≥ 48×48 on touch, ≥ 32 px with a pointer.

6. Phases

1. **Rebrand and adaptive layout** (Flutter): QuillFin name and app ids, brand tokens, light and dark theme, bundled fonts, the adaptive shell (bar / rail / sidebar) and list-detail panes, keyboard shortcuts, and golden tests at 360, 412, 600, 840, 1280 and 1600 widths.
2. **Convex backend**: schema, sync push/pull, entitlements and usage, the AI generation action (server key, plan limits), Stripe checkout, portal and webhook, auth config, and a vitest + convex-test suite.
3. **Flutter accounts and sync**: Clerk PKCE sign-in, secure token storage and refresh, local schema v3 (uid / updated_at / deleted_at / dirty), the sync engine, the Pro upgrade and manage flow, and Settings > Sync / Plan.
4. **Next.js web app**: the same screens and breakpoints on Convex's React client, Clerk components, and Stripe checkout from the web.
5. **Ship readiness**: setup guide (Clerk, Convex, Stripe step by step), environment config, CI for all three projects, privacy policy and terms placeholders, account deletion (required by both stores), app icons and splash, store listings.

Real keys are never committed. Publishable values come from build-time config (`--dart-define-from-file=config/<env>.json` , `.env.local` for web); secret values exist only as Convex environment variables.

7. Status

Phase 1 (rebrand and adaptive layout): done, 2026-10-27.

- **Renamed to QuillFin** on every platform:
  - Dart package `quillfin` .
  - App id `com.savepoint.quillfin` on Android, iOS, macOS and Linux; iOS and macOS projects added.
  - Windows exe `quillfin.exe` , AppImage `QuillFin` .
  - The local database is now `quillfin.db` ; desktop installs adopt an existing `quizapp.db` .
- **Placeholder brand** in `lib/theme/brand.dart` : light and dark palettes, Outfit and Inter, the drawn mark and SVGs, and a System/Light/Dark setting.
- **Adaptive shell**: a bottom bar on compact, a rail on medium and expanded or any short window, and a labelled sidebar on large and up.
- **Shortcuts**: `Ctrl/⌘+1…5` switch sections, `Ctrl/⌘+/` lists shortcuts, `Ctrl/⌘+S` saves a note, and in a quiz `Enter` submits and `1-4` rate.
- **Layouts**:
  - List-detail for the tracker and for notes inside a folder.
  - Sortable table view for the tracker on large windows.
  - Two-column Today.
  - Split sign-in.
  - Sectioned Settings.
  - Quiz ratings pinned to the bottom on phones.

    - Attempt logging opens as a dialog on wide or short windows.

- **Desktop windows** open at 1280×800 with a 360×560 minimum on Linux, Windows and macOS.
- **Tests**: layout tests at phone, landscape-phone, medium, tablet and desktop sizes (`test/screens/responsive_test.dart`). 318 tests, 100% line coverage of `lib/`.

Still open from §5 (not blocking the next phases):

- The extra-large three-pane notes layout (folder tree, list, editor). Notes currently use two panes from expanded up.
- Remembering window size and position: needs a window-management plugin.
- Right-click context menus and drag-to-move for notes.
- `Ctrl/⌘+N` and `Ctrl/⌘+K` (new note, search).
- Screenshot (golden) tests at each width. Layout is asserted with widget tests instead.

Phase 2 (Convex backend): done, 2026-10-28. See `backend.md`.

- **Schema** for users, the seven synced tables, subscriptions and usage. Synced rows mirror the local SQLite tables, with parent `uid`s.
- **Sync**: `sync:push` (last-writer-wins, clamped clocks, append-only history, tombstones, the size and row limits) and `sync:pull` (paged by `seq` across all tables).
- **Plans and usage**: limits in `plans.ts`, `billing:me`, and a monthly AI allowance that is reserved before the model call and refunded on failure.
- **AI generation** on QuillFin's Anthropic key, using the app's prompt.
- **Stripe**: Checkout, the Billing Portal, and a signature-checked webhook that always stores Stripe's current copy of the subscription.
- **Auth**: one Clerk issuer, accepting the web app's `convex` JWT and the Flutter app's OAuth ID token.
- **Tests**: 64 vitest + convex-test tests, 100% line coverage of `backend/convex/`, run in CI (`.github/workflows/backend.yml`).
- **Setup**: `setup_guide.md` covers the account steps. No real accounts exist yet; development uses an anonymous local Convex deployment.

Deferred from Phase 2:

- Account deletion (batched deletes, Stripe cancel, Clerk delete) moves to Phase 5 with the rest of the store requirements.
- Web-app read indexes move to Phase 4.

Phase 3 (Flutter accounts and sync): done, 2026-10-28. See `accounts_and_sync.md`.

- **Sign-in**: Clerk OAuth with PKCE through the system browser: a deep link on Android and iOS, a loopback listener on desktop. Tokens are kept in secure storage and the ID token is refreshed before it expires. "Continue on this device" stays, and is the only option in builds without cloud config and on the web build.
    - This replaces `flutter_web_auth_2` from §2, whose desktop support needs an embedded web view (and WebKitGTK on Linux). Doing the browser step in-house needs only `app_links` and `url_launcher`.

- **Local schema v3**: `uid`, `sync_updated_at` and `dirty` on every synced table, kept by SQLite triggers (including tombstones for cascades), so the repos are unchanged.
- **Sync engine**: push in batches of 200, then pull in pages. Newer dirty local rows win, children wait for their parents, and switching accounts re-sends everything. It runs on open, on resume, every 5 minutes, and within about 10 s of a change.
- **Settings > Sync and > Plan**, sign-out (keep or delete this device's copy), and generation on QuillFin's AI when signed in without a key. Upgrading is hidden on iOS.
- **Tests**: 394, 100% line coverage of `lib/`. There's also an opt-in live test that synced two devices through the real Convex backend, and the Linux integration test passes again.
- **Fixes found on the way**:
  - two FABs in the shell shared a hero tag, breaking page transitions;
  - the macOS release build had no network entitlement;
  - the integration test hadn't been updated for Phase 1's desktop window size.

Still open from Phase 3:

- Checks that need real accounts, listed in `setup_guide.md` §6: real Clerk sign-in, refresh, and Stripe test checkout.
- Notes pushed mid-edit aren't merged: last writer wins per row (§3).

Phase 4 (Next.js web app): done, 2026-10-28. See `webapp.md`.

- **The web app** (`webapp/`): Next.js 16, React 19, Tailwind 4, Clerk and Convex live queries. It has Today, Notes (folders, notes, questions, Generate), Quiz, Tracker (with the table view), Settings (Plan, Appearance, target recall) and Account.
  - Breakpoints and layouts match §5, the brand tokens live in `webapp/app/brand.css`, and the keyboard shortcuts match the app's.
  - Upgrades go through Stripe Checkout from the web.
- **Backend functions for it**: folders, notes, questions, problems and today. Every write goes through a shared `RowWriter`, so the apps pull web edits, and deletes cascade to tombstones.
- **One scheduler everywhere**: FSRS ported from the Dart package to TypeScript and checked against 36 sequences generated by the app's own code, identical to the millisecond. Grading is ported too.
- **Tests**: backend 131 (100% line coverage), web 31 (about 95% of component lines), Vitest 5 (Vitest 3 had advisories), and CI for the web app, which type-checks, tests and builds.

Not built for the web (the app has them): file import, your own AI key, data export, and offline use. The web is online-only by design.

Phase 5 (ship readiness): in progress, 2026-11-06. See `store_listing.md`.

- **Account deletion** (both stores require it): `account:deleteAccount` deletes the Stripe customer (cancelling Pro), then the Clerk user, then every row in batches. It is offered under Account in the app and on the web account page.
- **Legal**: terms, subscriptions, privacy, cookies and account deletion at savepoint.co.nz/legal/quillfin, linked from the app and the web app. Minimum age 13.
- **Store builds sell nothing**: iOS and Android show the plan but no upgrade or manage buttons, and no pointer to the website. This replaces the §2 idea of linking Android to Stripe, which Play's billing policy

doesn't allow.

- **Icons and graphics** for every platform from the brand SVG (`tool/build_icons.mjs`), Play release signing from `key.properties`, and `ITSAppUsesNonExemptEncryption` on iOS.
- **Hosting**: the web build moved from alotanotes.savepoint.co.nz to quillfin.savepoint.co.nz.

Still open: the store accounts and listings and production keys. The iPhone release comes later (decided 2026-11-06): Android, the web and desktop launch first, and Pro gets Apple In-App Purchase before the App Store release.